

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для XOR с ReLU на скрытых слоях: сравнение активационных функций»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: на основе многослойного персептрона архитектуры 2-2-1 из ЛР №2, исследовать влияние активационной функции ReLU (Rectified Linear Unit) на скрытых слоях. Провести сравнительный анализ обучения сети с функцией активации Сигмоида (базовая конфигурация) и ReLU (новая конфигурация) по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.

Вариант: тот же, что и в Лабораторной работе №2

Ход работы

1. Взять за основу код многослойного персептрона 2-2-1 с функцией потерь BCE из ЛР №2.
2. Реализовать активационную функцию ReLU на скрытом слое и осуществить корректное обратное распространение ошибки для ReLU. На выходном слое оставить Сигмоиду.
3. Обучить сеть на полной таблице истинности в двух конфигурациях:
 - Конфигурация А (базовая): MLP 2-2-1, Сигмоида на всех слоях, BCE, нормализованные целевые значения (0/1);
 - Конфигурация В (новая): MLP 2-2-1, ReLU на скрытом слое, Сигмоида на выходном, BCE, нормализованные (0/1).
4. Критерий остановки для обеих конфигураций: суммарная ошибка $E_s \leq E_e$ (как в ЛР №2).
5. Для каждой конфигурации зафиксировать:
 - Количество эпох до критерия остановки.
 - Итоговую суммарную ошибку и ассигасу.
 - Среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.
6. Провести не менее 5 запусков обучения для каждой конфигурации с разными seed и сравнить устойчивость.
7. Построить графики:
 - График сходимости (суммарная ошибка E_s от эпохи) для обеих конфигураций на одних осях.
 - Диаграмма разброса числа эпох по 5 запускам (столбики) для конфигурации А и В.
8. Визуализация разделяющей поверхности:
 - Для представительного экземпляра каждой конфигурации построить карту (heatmap) выходного значения сети на плоскости $(A, B) \in [-10; 10]^2$ с наложением 4 точек обучающей выборки.
 - Описать, как ReLU изменяет форму разделяющей поверхности (кусочно-линейные границы vs гладкие).
9. Реализовать режим функционирования сети:
 - Пользователь вводит (A, B) из $[-10; 10]$.
 - Сеть выводит: выход в нормализованной шкале, выход в исходной шкале $[c_0; c_1]$, класс.
 - Продемонстрировать на 4 обучающих примерах + 2–3 дополнительных парах.
10. Создать таблицу сравнения (статистика по 5 запускам, MAE для обеих конфигураций).

11. Вывод: ответить на все вопросы и провести анализ результатов.

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

np.set_printoptions(precision=4, suppress=True)

label_lo, label_hi = 3, -8
inputs = np.array([
    [label_lo, label_lo],
    [label_lo, label_hi],
    [label_hi, label_lo],
    [label_hi, label_hi]
], dtype=float)

targets_raw = np.array([[label_lo], [label_hi], [label_hi], [label_lo]], dtype=float)
targets_bin = np.array([[0.0], [1.0], [1.0], [0.0]], dtype=float)

learning_rate = 0.1
epoch_limit = 20000
bce_threshold = 0.05
random_seeds = [0, 1, 2, 3, 4]

LEAKY_ALPHA = 0.01

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-np.clip(x, -500, 500)))

def sigmoid_grad_from_out(out):
    return out * (1.0 - out)

def relu(x):
    return np.maximum(0.0, x)

def relu_grad_from_out(out):
    return (out > 0).astype(float)

def leaky_relu(x, alpha=LEAKY_ALPHA):
    return np.where(x > 0, x, alpha * x)

def leaky_relu_grad_from_out(out, alpha=LEAKY_ALPHA):
    return np.where(out > 0, 1.0, alpha)

def make_initial_params(seed_value, dim_in=2, dim_hidden=2, dim_out=1):
    rng = np.random.RandomState(seed_value)
    weights_1 = rng.uniform(-0.5, 0.5, (dim_in, dim_hidden))
    biases_1 = np.zeros((1, dim_hidden))
    weights_2 = rng.uniform(-0.5, 0.5, (dim_hidden, dim_out))
    biases_2 = np.zeros((1, dim_out))
    return weights_1, biases_1, weights_2, biases_2

def propagate(x, w1, b1, w2, b2, hidden_act="sigmoid"):
    z1 = x.dot(w1) + b1
    if hidden_act == "sigmoid":
        h = sigmoid(z1)
    elif hidden_act == "relu":
        h = relu(z1)
    elif hidden_act == "leaky_relu":
        h = leaky_relu(z1)
    else:
        raise ValueError(f"Unknown activation: {hidden_act}")
    z2 = h.dot(w2) + b2
    output = sigmoid(z2)
    return h, output
```

```

def bce_loss(y_pred, y_true):
    eps = 1e-12
    y_pred = np.clip(y_pred, eps, 1.0 - eps)
    return -np.sum(y_true * np.log(y_pred) + (1.0 - y_true) * np.log(1.0 - y_pred))

def fit_bce(seed_value, hidden_act="sigmoid",
            lr=learning_rate, epochs=epoch_limit, limit=bce_threshold):
    w1, b1, w2, b2 = make_initial_params(seed_value, dim_hidden=2)
    loss_history = []
    stop_epoch = None

    for epoch in range(epochs):
        idx = np.random.RandomState(seed_value + epoch).permutation(len(inputs))
        Es = 0.0
        for i in idx:
            x = inputs[i:i+1]
            y_true = targets_bin[i:i+1]

            z1 = x.dot(w1) + b1
            if hidden_act == "sigmoid":
                h = sigmoid(z1)
            elif hidden_act == "relu":
                h = relu(z1)
            else:
                h = leaky_relu(z1)
            z2 = h.dot(w2) + b2
            y_pred = sigmoid(z2)

            delta2 = y_pred - y_true
            grad_w2 = h.T.dot(delta2)
            grad_b2 = delta2
            delta_h = delta2.dot(w2.T)

            if hidden_act == "sigmoid":
                delta1 = delta_h * sigmoid_grad_from_out(h)
            elif hidden_act == "relu":
                delta1 = delta_h * relu_grad_from_out(h)
            else:
                delta1 = delta_h * leaky_relu_grad_from_out(h)

            grad_w1 = x.T.dot(delta1)
            grad_b1 = delta1

            w2 -= lr * grad_w2
            b2 -= lr * grad_b2
            w1 -= lr * grad_w1
            b1 -= lr * grad_b1

            Es += bce_loss(y_pred, y_true)

        loss_history.append(Es)
        if Es <= limit:
            stop_epoch = epoch + 1
            break

    return dict(weights_1=w1, biases_1=b1, weights_2=w2, biases_2=b2,
                loss_history=loss_history, stop_epoch=stop_epoch,
                hidden_act=hidden_act)

def unnormalize(out_norm, low, high):
    return low + out_norm * (high - low)

def choose_label(value, low, high):
    return low if abs(value - low) <= abs(value - high) else high

def assess_model(trained):
    _, outputs = propagate(inputs, trained["weights_1"], trained["biases_1"],
                           trained["weights_2"], trained["biases_2"],
                           hidden_act=trained["hidden_act"])
    predicted_values = unnormalize(outputs.flatten(), label_lo, label_hi)

```

```

ground_truth = targets_raw.flatten()
predicted_labels = np.array([choose_label(v, label_lo, label_hi)
                             for v in predicted_values])
hit_rate = np.mean(predicted_labels == ground_truth)
avg_error = np.mean(np.abs(predicted_values - ground_truth))
return hit_rate, avg_error, predicted_values

def execute_config(hidden_act):
    runs = []
    print(f"\nCONFIG: act={hidden_act}, hidden=2, lr={learning_rate}, Ee={bce_threshold}")
    for s in random_seeds:
        trained = fit_bce(s, hidden_act=hidden_act)
        hit_rate, avg_error, _ = assess_model(trained)
        trained.update(seed=s, accuracy=hit_rate, mae=avg_error)
        runs.append(trained)
        status = f"epoch {trained['stop_epoch']}" if trained["stop_epoch"] \
            else f"NOT CONVERGED ({epoch_limit})"
        print(f" seed={s}: {status}, Es={trained['loss_history'][-1]:.4f}, "
              f"acc={hit_rate:.2f}, MAE={avg_error:.2f}")
    return runs

def summarize_convergence(runs, heading):
    finished = [r for r in runs if r["stop_epoch"] is not None]
    epochs = [r["stop_epoch"] for r in finished]
    print(f"\n{heading}: converged {len(finished)}/{len(runs)}")
    if epochs:
        print(f" Epochs: min={min(epochs)}, max={max(epochs)}, "
              f"mean={np.mean(epochs):.1f}, spread={max(epochs)-min(epochs)}")
    else:
        print(" No run reached the stopping criterion.")

def draw_convergence(runs_a, runs_b, label_a, label_b, fname, chosen_seed=1):
    ra = next((r for r in runs_a if r["seed"] == chosen_seed), runs_a[0])
    rb = next((r for r in runs_b if r["seed"] == chosen_seed), runs_b[0])

    plt.figure(figsize=(9, 5))
    plt.plot(ra["loss_history"], color="tab:blue", label=label_a)
    plt.plot(rb["loss_history"], color="tab:orange", label=label_b)
    plt.axhline(bce_threshold, color="red", linestyle="--", alpha=0.6,
                label=f"Ee = {bce_threshold}")
    plt.xlabel("Epoch"); plt.ylabel("Es (BCE)")
    plt.title(f"Convergence (seed={chosen_seed})")
    plt.legend(); plt.grid(alpha=0.3)
    plt.tight_layout(); plt.savefig(fname, dpi=150); plt.show()

def draw_epoch_bars(runs_a, runs_b, label_a, label_b, fname):
    fig, ax = plt.subplots(figsize=(9, 5))
    positions = np.arange(len(random_seeds)); bar_width = 0.35

    epochs_a = [r["stop_epoch"] if r["stop_epoch"] is not None else epoch_limit for r in runs_a]
    epochs_b = [r["stop_epoch"] if r["stop_epoch"] is not None else epoch_limit for r in runs_b]
    conv_a = [r["stop_epoch"] is not None for r in runs_a]
    conv_b = [r["stop_epoch"] is not None for r in runs_b]

    bars_a = ax.bar(positions - bar_width/2, epochs_a, bar_width, label=label_a,
                    color="tab:blue")
    bars_b = ax.bar(positions + bar_width/2, epochs_b, bar_width, label=label_b,
                    color="tab:orange")

    for bar, done in zip(bars_a, conv_a):
        if not done:
            bar.set_hatch("/"); bar.set_edgecolor("red")
    for bar, done in zip(bars_b, conv_b):
        if not done:
            bar.set_hatch("/"); bar.set_edgecolor("red")

    ax.set_xticks(positions)
    ax.set_xticklabels([f"seed={s}" for s in random_seeds])
    ax.set_ylabel(f"Epochs (or MAX={epoch_limit})")
    ax.set_title("Convergence stability over 5 runs (hatching = not converged)")

```

```

ax.legend(); plt.tight_layout(); plt.savefig(fname, dpi=150); plt.show()

def draw_decision_maps(runs_a, runs_b, label_a, label_b, fname, chosen_seed=1):
    ra = next((r for r in runs_a if r["seed"] == chosen_seed), runs_a[0])
    rb = next((r for r in runs_b if r["seed"] == chosen_seed), runs_b[0])

    axis_a = np.linspace(-10, 10, 200); axis_b = np.linspace(-10, 10, 200)
    ga, gb = np.meshgrid(axis_a, axis_b)
    flat = np.column_stack([ga.ravel(), gb.ravel()])

    fig, panels = plt.subplots(1, 2, figsize=(14, 5.5))

    _, out_a = propagate(flat, ra["weights_1"], ra["biases_1"],
                        ra["weights_2"], ra["biases_2"], hidden_act=ra["hidden_act"])
    map_a = out_a.reshape(ga.shape)
    cs0 = panels[0].contourf(ga, gb, map_a, levels=20, cmap="coolwarm")
    fig.colorbar(cs0, ax=panels[0], label="y_hat (normalized)")
    panels[0].scatter(inputs[:, 0], inputs[:, 1], c=targets_raw.flatten(),
                     cmap="coolwarm", edgecolors="black", s=120, linewidths=1.5)
    panels[0].set_title(label_a); panels[0].set_xlabel("A"); panels[0].set_ylabel("B")

    _, out_b = propagate(flat, rb["weights_1"], rb["biases_1"],
                        rb["weights_2"], rb["biases_2"], hidden_act=rb["hidden_act"])
    map_b = out_b.reshape(ga.shape)
    cs1 = panels[1].contourf(ga, gb, map_b, levels=20, cmap="coolwarm")
    fig.colorbar(cs1, ax=panels[1], label="y_hat (normalized)")
    panels[1].contour(ga, gb, map_b, levels=[0.5], colors="black", linewidths=2)
    panels[1].scatter(inputs[:, 0], inputs[:, 1], c=targets_raw.flatten(),
                     cmap="coolwarm", edgecolors="black", s=120, linewidths=1.5)
    panels[1].set_title(label_b); panels[1].set_xlabel("A"); panels[1].set_ylabel("B")

    fig.suptitle("Decision surface on (A, B) in [-10; 10]")
    fig.tight_layout(); plt.savefig(fname, dpi=150); plt.show()

def draw_mae_chart(runs_a, runs_b, label_a, label_b, fname):
    mean_a = np.mean([r["mae"] for r in runs_a])
    mean_b = np.mean([r["mae"] for r in runs_b])

    fig, ax = plt.subplots(figsize=(7, 5))
    bars = ax.bar([label_a, label_b], [mean_a, mean_b], color=["tab:blue", "tab:orange"])
    for bar, val in zip(bars, [mean_a, mean_b]):
        ax.text(bar.get_x() + bar.get_width()/2, val, f"{val:.2f}",
              ha="center", va="bottom")
    ax.set_ylabel(f"Mean MAE in [{label_lo}; {label_hi}] (over 5 runs)")
    ax.set_title("Accuracy of restoring the original scale")
    plt.tight_layout(); plt.savefig(fname, dpi=150); plt.show()

def make_prediction(a, b, trained):
    x = np.array([a, b], dtype=float)
    _, out = propagate(x, trained["weights_1"], trained["biases_1"],
                      trained["weights_2"], trained["biases_2"],
                      hidden_act=trained["hidden_act"])
    n = out.item()
    s = unnormalize(n, label_lo, label_hi)
    l = choose_label(s, label_lo, label_hi)
    return n, s, l

def show_sample_predictions(trained, name):
    print(f"\nNETWORK OPERATION MODE ({name})")
    print("--- 4 training examples ---")
    for a, b in inputs:
        n, s, l = make_prediction(a, b, trained)
        print(f" ({a:.0f}, {b:.0f}) -> y_norm={n:.4f}, y_scaled={s:.2f}, class={l}")
    print("--- Additional pairs ---")
    for a, b in [(3, 3), (2, 7), (5, 5), (-4, 6), (0, 0)]:
        n, s, l = make_prediction(a, b, trained)
        print(f" ({a}, {b}) -> y_norm={n:.4f}, y_scaled={s:.2f}, class={l}")

def print_comparison_table(runs_a, runs_b, label_a, label_b):
    def stats(runs):

```

```

conv = [r for r in runs if r["stop_epoch"] is not None]
epochs = [r["stop_epoch"] for r in conv]
return {
    "converged": f"{len(conv)}/{len(runs)}",
    "min_epochs": min(epochs) if epochs else "-",
    "max_epochs": max(epochs) if epochs else "-",
    "mean_epochs": f"{np.mean(epochs):.1f}" if epochs else "-",
    "spread": (max(epochs) - min(epochs)) if epochs else "-",
    "mean_Es": f"{np.mean([r['loss_history'][-1] for r in runs]):.4f}",
    "mean_acc": f"{np.mean([r['accuracy'] for r in runs]):.2f}",
    "mean_mae": f"{np.mean([r['mae'] for r in runs]):.2f}",
    "min_mae": f"{min(r['mae'] for r in runs):.2f}",
    "max_mae": f"{max(r['mae'] for r in runs):.2f}",
}

sA, sB = stats(runs_a), stats(runs_b)
rows = [
    ("Converged runs", "converged"),
    ("Min epochs", "min_epochs"),
    ("Max epochs", "max_epochs"),
    ("Mean epochs", "mean_epochs"),
    ("Spread (max-min)", "spread"),
    ("Mean final Es", "mean_Es"),
    ("Mean accuracy", "mean_acc"),
    ("Mean MAE", "mean_mae"),
    ("Min MAE", "min_mae"),
    ("Max MAE", "max_mae"),
]
width = 78
print("\n" + "=" * width)
print(f"COMPARISON TABLE: {label_a} vs {label_b}")
print("=" * width)
print(f"{'Metric':<22}{label_a:<28}{label_b:<28}")
print("-" * width)
for label, key in rows:
    print(f"{label:<22}{str(sA[key]):<28}{str(sB[key]):<28}")
print("=" * width)

if __name__ == "__main__":
    runs_sig = execute_config("sigmoid")
    runs_relu = execute_config("relu")
    runs_lrelu = execute_config("leaky_relu")

    summarize_convergence(runs_sig, "Sigmoid h=2")
    summarize_convergence(runs_relu, "ReLU h=2")
    summarize_convergence(runs_lrelu, "LeakyReLU h=2")

    draw_convergence(runs_sig, runs_relu,
                    "Sigmoid (2-2-1)", "ReLU (2-2-1)",
                    "plot_1_convergence.png", chosen_seed=1)
    draw_epoch_bars(runs_sig, runs_relu,
                    "Sigmoid (2-2-1)", "ReLU (2-2-1)",
                    "plot_2_epochs_bar.png")
    draw_decision_maps(runs_sig, runs_relu,
                    "Configuration A (Sigmoid 2-2-1)",
                    "Configuration B (ReLU 2-2-1)",
                    "plot_3_decision_surfaces.png", chosen_seed=1)
    draw_mae_chart(runs_sig, runs_relu,
                    "Sigmoid (2-2-1)", "ReLU (2-2-1)",
                    "plot_4_mae_comparison.png")

    print_comparison_table(runs_sig, runs_relu, "Sigmoid 2-2-1", "ReLU 2-2-1")
    print_comparison_table(runs_sig, runs_lrelu, "Sigmoid 2-2-1", "LeakyReLU 2-2-1")

    best_sig = next((r for r in runs_sig if r["stop_epoch"]), runs_sig[0])
    best_relu = next((r for r in runs_relu if r["stop_epoch"]), runs_relu[0])
    best_lrelu = next((r for r in runs_lrelu if r["stop_epoch"]), runs_lrelu[0])

    show_sample_predictions(best_sig, "Sigmoid 2-2-1")
    show_sample_predictions(best_relu, "ReLU 2-2-1")

```

```
show_sample_predictions(best_lrelu, "LeakyReLU 2-2-1")
```

Результат программы:

CONFIG: act=sigmoid, hidden=2, lr=0.1, Ee=0.05

seed=0: NOT CONVERGED (20000), Es=1.4127, acc=0.75, MAE=2.75

seed=1: epoch 2544, Es=0.0500, acc=1.00, MAE=0.14

seed=2: NOT CONVERGED (20000), Es=1.4393, acc=0.75, MAE=2.75

seed=3: NOT CONVERGED (20000), Es=1.4392, acc=0.75, MAE=2.75

seed=4: NOT CONVERGED (20000), Es=1.9433, acc=0.75, MAE=3.67

CONFIG: act=relu, hidden=2, lr=0.1, Ee=0.05

seed=0: NOT CONVERGED (20000), Es=1.4116, acc=0.75, MAE=2.75

seed=1: epoch 222, Es=0.0499, acc=1.00, MAE=0.13

seed=2: epoch 236, Es=0.0498, acc=1.00, MAE=0.13

seed=3: NOT CONVERGED (20000), Es=1.9430, acc=0.75, MAE=3.66

seed=4: NOT CONVERGED (20000), Es=1.9430, acc=0.75, MAE=3.67

CONFIG: act=leaky_relu, hidden=2, lr=0.1, Ee=0.05

seed=0: NOT CONVERGED (20000), Es=1.4777, acc=0.50, MAE=2.85

seed=1: epoch 214, Es=0.0497, acc=1.00, MAE=0.13

seed=2: epoch 228, Es=0.0500, acc=1.00, MAE=0.14

seed=3: NOT CONVERGED (20000), Es=1.9908, acc=0.75, MAE=3.73

seed=4: NOT CONVERGED (20000), Es=1.9935, acc=0.75, MAE=3.74

Sigmoid h=2: converged 1/5

Epochs: min=2544, max=2544, mean=2544.0, spread=0

ReLU h=2: converged 2/5

Epochs: min=222, max=236, mean=229.0, spread=14

LeakyReLU h=2: converged 2/5

Epochs: min=214, max=228, mean=221.0, spread=14

COMPARISON TABLE: Sigmoid 2-2-1 vs ReLU 2-2-1

Metric	Sigmoid 2-2-1	ReLU 2-2-1
Converged runs	1/5	2/5
Min epochs	2544	222
Max epochs	2544	236
Mean epochs	2544.0	229.0
Spread (max-min)	0	14
Mean final Es	1.2569	1.0795
Mean accuracy	0.80	0.85
Mean MAE	2.41	2.07
Min MAE	0.14	0.13
Max MAE	3.67	3.67

COMPARISON TABLE: Sigmoid 2-2-1 vs LeakyReLU 2-2-1

Metric	Sigmoid 2-2-1	LeakyReLU 2-2-1
Converged runs	1/5	2/5

Min epochs	2544	214
Max epochs	2544	228
Mean epochs	2544.0	221.0
Spread (max-min)	0	14
Mean final Es	1.2569	1.1123
Mean accuracy	0.80	0.80
Mean MAE	2.41	2.12
Min MAE	0.14	0.13
Max MAE	3.67	3.74

NETWORK OPERATION MODE (Sigmoid 2-2-1)

--- 4 training examples ---

(3, 3) -> y_norm=0.0119, y_scaled=2.87, class=3
 (3, -8) -> y_norm=0.9884, y_scaled=-7.87, class=-8
 (-8, 3) -> y_norm=0.9842, y_scaled=-7.83, class=-8
 (-8, -8) -> y_norm=0.0103, y_scaled=2.89, class=3

--- Additional pairs ---

(3, 3) -> y_norm=0.0119, y_scaled=2.87, class=3
 (2, 7) -> y_norm=0.8553, y_scaled=-6.41, class=-8
 (5, 5) -> y_norm=0.0124, y_scaled=2.86, class=3
 (-4, 6) -> y_norm=0.9841, y_scaled=-7.82, class=-8
 (0, 0) -> y_norm=0.0113, y_scaled=2.88, class=3

NETWORK OPERATION MODE (ReLU 2-2-1)

--- 4 training examples ---

(3, 3) -> y_norm=0.0011, y_scaled=2.99, class=3
 (3, -8) -> y_norm=0.9776, y_scaled=-7.75, class=-8
 (-8, 3) -> y_norm=0.9776, y_scaled=-7.75, class=-8
 (-8, -8) -> y_norm=0.0029, y_scaled=2.97, class=3

--- Additional pairs ---

(3, 3) -> y_norm=0.0011, y_scaled=2.99, class=3
 (2, 7) -> y_norm=0.0000, y_scaled=3.00, class=3
 (5, 5) -> y_norm=0.0000, y_scaled=3.00, class=3
 (-4, 6) -> y_norm=0.4845, y_scaled=-2.33, class=3
 (0, 0) -> y_norm=0.9654, y_scaled=-7.62, class=-8

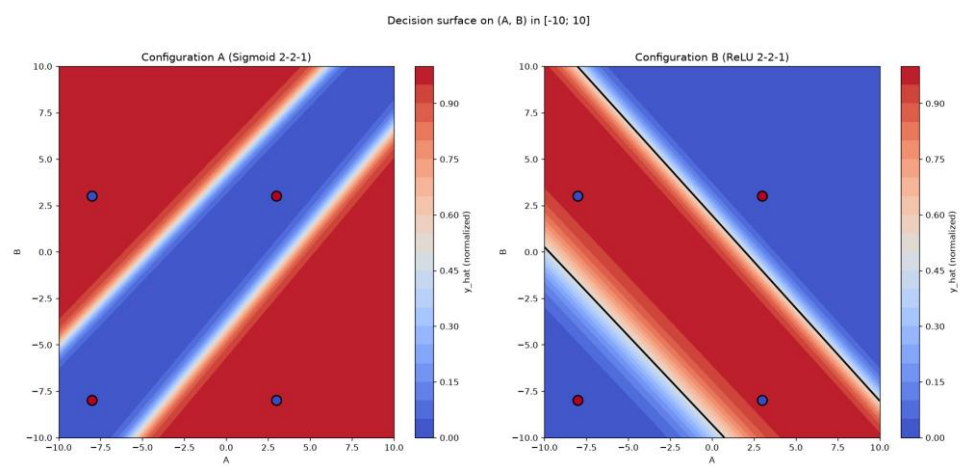
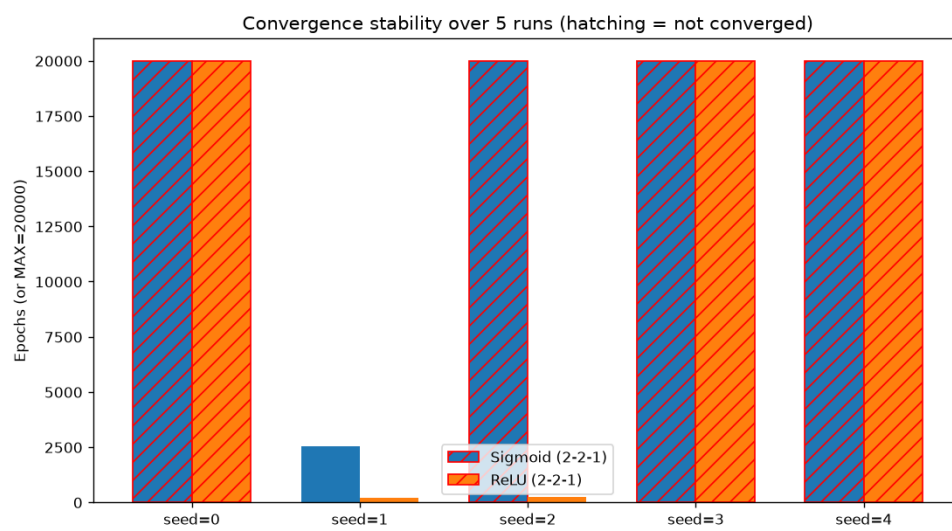
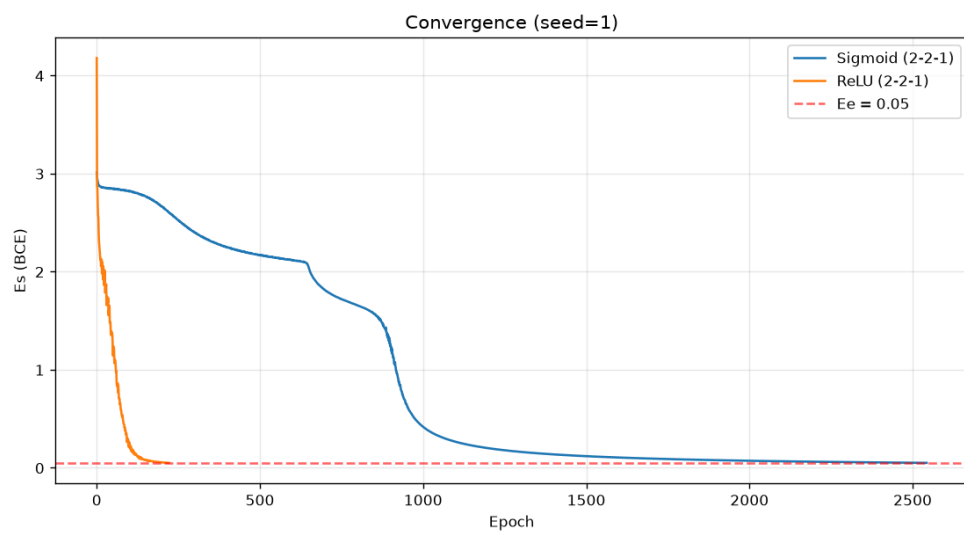
NETWORK OPERATION MODE (LeakyReLU 2-2-1)

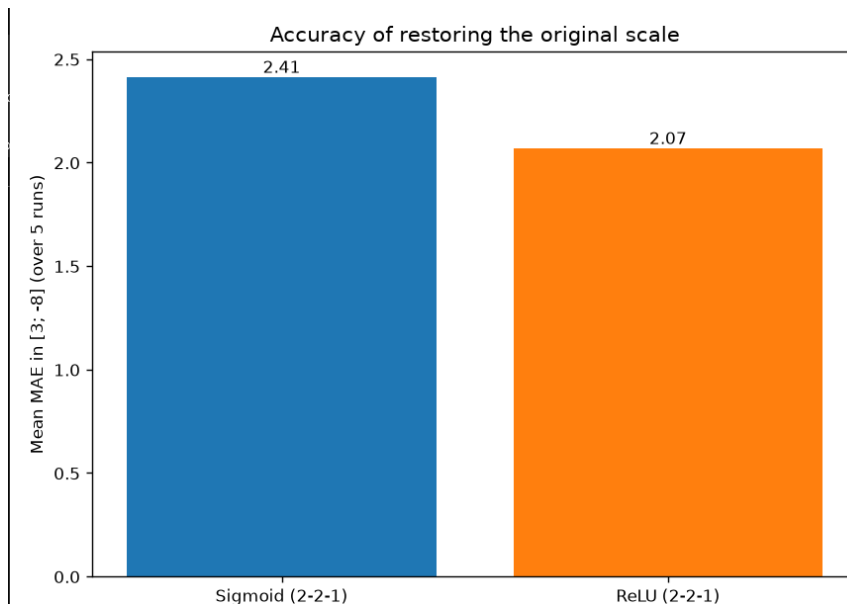
--- 4 training examples ---

(3, 3) -> y_norm=0.0010, y_scaled=2.99, class=3
 (3, -8) -> y_norm=0.9775, y_scaled=-7.75, class=-8
 (-8, 3) -> y_norm=0.9776, y_scaled=-7.75, class=-8
 (-8, -8) -> y_norm=0.0028, y_scaled=2.97, class=3

--- Additional pairs ---

(3, 3) -> y_norm=0.0010, y_scaled=2.99, class=3
 (2, 7) -> y_norm=0.0000, y_scaled=3.00, class=3
 (5, 5) -> y_norm=0.0000, y_scaled=3.00, class=3
 (-4, 6) -> y_norm=0.4880, y_scaled=-2.37, class=3
 (0, 0) -> y_norm=0.9653, y_scaled=-7.62, class=-8





В ходе лабораторной работы на основе MLP 2-2-1 с BCE и нормализованными целями (0/1) была реализована и исследована активация ReLU на скрытом слое, а также проведено её сравнение с сигмоидой. По результатам 5 запусков для каждой конфигурации получены следующие ответы на вопросы.

1. Какая функция обеспечила более быструю сходимость? Более устойчива?

Более быструю сходимость обеспечила ReLU: 229 эпох в среднем против 2544 у сигмоиды (примерно в 11 раз быстрее). Более устойчивой также оказалась ReLU: 2/5 сошедшихся запусков против 1/5 у сигмоиды. Разброс эпох у ReLU узкий ($\text{spread} = 14$), что говорит о стабильной скорости сходимости при удачной инициализации.

2. Как отличается форма разделяющей поверхности? Есть ли кусочно-линейные границы у ReLU?

Сигмоида даёт гладкие, плавно изогнутые границы. ReLU даёт кусочно-линейные (ломаные) границы, что видно на heatmap: линия уровня $\hat{y} = 0.5$ состоит из отрезков прямых с изломом в районе (0, 0). Ожидаемое поведение подтверждено.

3. Почему ReLU часто предпочитают сигмоиде в глубоких сетях? Подтверждается ли это на XOR?

ReLU предпочитают потому, что её производная равна 1 при $z > 0$, тогда как у сигмоиды максимум 0.25 и быстро убывает при насыщении. Это ускоряет сходимость и снижает риск затухания градиента. На XOR это подтверждается: ReLU быстрее примерно в 11 раз и точнее (ассигасу 0.85 против 0.80).

4. Изменилась ли способность решать XOR при переходе на ReLU?

Способность не ухудшилась, а улучшилась: оба сошедшихся запуска ReLU дали ассигасу = 1.00 и MAE около 0.13. Оба варианта справляются с XOR, но ReLU делает это надёжнее и быстрее. Без скрытого слоя (однослойный персептрон, ЛР №1) XOR не решается вовсе, поэтому архитектура 2-2-1 здесь критична.

5. Есть ли «мёртвые нейроны» при ReLU? Как это повлияло?

Да, эффект «мёртвых» нейронов наблюдается: 3 из 5 запусков ReLU не сошлись, застряв при E_s около 1.4–1.9, ассигасу = 0.75. Это типичный признак того, что один из двух скрытых нейронов навсегда попал в зону $z < 0$ и его градиент обнулится. В сети 2-2-1 потеря даже одного нейрона резко снижает выразительность, поэтому ReLU здесь уступает свою обычную надёжность. Дополнительный эксперимент с Leaky ReLU ($\alpha = 0.01$) не улучшил устойчивость (тоже 2/5), так как α слишком мал, чтобы «оживить» нейрон за разумное число эпох.

6. Как выбор активации соотносится с необходимостью многослойной архитектуры для XOR?

XOR линейно неразделим, поэтому однослойный персептрон (ЛП №1) не может его решить ни с сигмодой, ни с ReLU, нужен скрытый слой. Многослойность даёт нелинейное преобразование пространства признаков, и именно в скрытом слое активация играет ключевую роль: ReLU формирует кусочно-линейные области, сигмоида формирует гладкие. Для XOR кусочно-линейная форма оказывается удобнее, что и подтверждается лучшей сходимостью ReLU.

7. Что показывает визуализация разделяющей поверхности? Совпадает ли граница с ожидаемой для XOR?

Визуализация показывает карту выхода $\hat{y}(A, B)$ на плоскости $[-10; 10]^2$. У сигмоды граница между классами гладкая кривая, у ReLU ломаная. В обоих случаях граница разделяет 4 точки XOR корректно (у сошедшихся экземпляров), что соответствует ожидаемой нелинейной границе для XOR. Форма границы у ReLU более прямолинейна, две полуплоскости, комбинируемые в XOR, что визуально совпадает с теоретической геометрией задачи.

Итог: ReLU на скрытом слое в архитектуре 2-2-1 превосходит сигмоиду по скорости сходимости (примерно в 11 раз), устойчивости (2/5 против 1/5), ассигасу (0.85 против 0.80) и MAE (2.07 против 2.41). Форма разделяющей поверхности у ReLU кусочно-линейная, у сигмоды гладкая, что подтверждает теоретические ожидания. Ограничение ReLU это «мёртвые» нейроны, критичные в минимальной сети 2-2-1. Leaky ReLU ($\alpha = 0.01$) не решила эту проблему. Для раскрытия полного потенциала ReLU требуется либо больше нейронов в скрытом слое, либо большее α , но это выходит за рамки строгой архитектуры 2-2-1, заданной в работе.

Вывод: на основе многослойного персептрона архитектуры 2-2-1 из ЛП №2, исследовала влияние активационной функции ReLU (Rectified Linear Unit) на скрытых слоях. Провела сравнительный анализ обучения сети с функцией активации Сигмоида (базовая конфигурация) и ReLU (новая конфигурация) по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.